1. Repartition vs Coalesce

- What is partition?
  A partition is a unit of parallel data processing.
  More partitions → More parallelism.

- Why Partition :- For performance, for parallel processing ~~for~~ for distributed computing

There are two important concepts
1. Choosing right number of partitions
2. Choosing the right size of the partition

Numbers
Suppose there are 16 cores (Executors) then number of partition should be 16 or multiple of 16 for better utilization of all executors and

Size
Evenly distributed partition improves the performance.
If we have one partition of 10 MB and other one of 1 GB then first one will finish its task quickly at ~~cost~~ its idealy. So overall application will take time while processing 1 GB file.

Default partitions of RDD/DF

- The parameter sc.defaultParallelism determines the number of partitions when creating data within Spark. Default value is 8

- When reading data from external system partitions are created by on parameters
  spark.sql.files.maxPartitionBytes. (Default 128 MB)

- To read a file and to create partition while reading it, the file must be is splitable format.

CSV + Gzip compression is not splittable but Parquet + Snappy is ~~com~~ splittable

- **Repartition**

What it is?

Repartition is pspark function used to increase or decrease number of partitions

Repartition alway shuffle the data and build new partitions from scratch

- Shuffle is one of the costlest operation in spark

Good part about repartion is that it create partions in almost equal sizes.

- **Coalesce**
  - Coalesce is only used for decreasing the number of partition.
  - Coalesce doesn't require a full shuffle.
  - So Coalesce combines few partitions, or we can say ~~partitions~~ it shuffle data only from few partitions.

Varified ? check

- Due to paution merge, it produces uneven size of pautions.

2. Cache vs Persist

- Cache is programming mechanism which gives an option to store the data in memory ~~a~~ across nodes.
- Persist is programming mechanism which gives an option to store the data either in memory or in disc across nodes.

Why Cache / Persist :-

If we are using a dataframe multiple time, Spark will calculate that dataframe everytime.
If we can store the df then Spark will use it. and there will be increase in peformance.

Example

```
df1 = Spark.read.CSV("path")
df2 = df.filter( )
df3 = df2.withColumn( )
df4 = df2.withColumn( )
df5 = df2.withColumn( )
```

Here we are using df2 again and again and suppose that these operation has some action in middle also, so it better to store df2 dataframe so that we dont need to calculate it againe again.

- So if we can improve the performance by caching the df2 dataframe Why don't we store all the dataframe then.

1. If it is used only one time then there is no use of storing it.

2. Storing all the df's will take lots of memory from the executors, which will later raise the

- Cache always store data in memory and never in disk.

- Persist can store data either in memory or in disk or in both memory and disk.

Syntax of Persist()
df.persist()

With persist we can choose
- Memory or disk (where to store)
- Serialized Vs Deserialized (How data is stored)
- Replication

df.persist(StorageLevel.MEMORY_ONLY)

What is only memory more than memory size.

- Different Storage Levels.

1. MEMORY_ONLY :- Same as Cache.
2. MEMORY_ONLY_SER :- By Persisting the RDD in a Serialized (binary) form help to reduce the size of the RDD, thus making space for more RDD to be persisted in cache memory. Space efficient but not time-efficient

3. MEMORY_AND_DISK :- Store partitions in Disk which not fit in memory.

4. Memory and Disk_SER :- Both in memory in disk but with SERialized.

5. DISK_ONLY :- Persist data in disk only, which would require network input and output operation thus time consuming. But still better than re-computation each time through DAG.

DISK_ONLY_2
MEMORY_AND_DISK_2
MEMORY_ONLY_2
MEMORY AND DISK ONLY_SER 2.

Same as above but it create replication in another node so. if any failure of node, still data is accessible through it

- **What happens If Data > Memory while persist or Cache?**

When data doesn't fit, Spark does not spill to disk,

Some partitions are caches in memory and others are not caches.
When we reuse the dataframe, Chached partition operations are fast, Unchached recomputed from lineage.

Ref PT to Notion Future

3. Broad Variable

It is programming mechanism in Spark, through which we can keep read-only copy of data into each node of the cluster instead of sending it to every time a task needs it.

- A read-only copy of data sent to every executor, stored locally on each node.

broadcast_val = SparkSparkContext.broadcast(my_dict)

What happens.

1. Driver has my_dict
2. Spark Serializes it
3. Sends it to executors using TorrentBroadcast
4. Each executor store a local copy in memory

- It avoids data shuffle thus improve peformace
- It is ideal for joining situation where one side of join is tiny table.

- Only suitable for smaller tables as it gets cached in each node. If created for larger tables, it would consume memory of each of worker node thus hitting the performance.

- The size of the broadcast variable should fit into drives memory otherwise leading to OOM issues.

```
from pyspark.sql.functions import broadcast
joindf = df.join(broadcast(df2), "store_id", "inner")
joindf.explain()
```